CS 2840 Fall 2025
10/23/2025

Scribes: Tim Guan and Richard Zhu
Based on notes by: Nico Courty

Lecture 16: Nico Courty – Domain Adaptation

Contents

1 Motivating Domain Adaptation **1**

2 Intro to Optimal Transport (OT) **2**

3 Optimal Transport for Domain Adaptation via Barycentric Mapping **4**

4 OT on Joint Distributions (JDOT) **5**

1 Motivating Domain Adaptation

OT background - covered in depth later

Reviewed: Monge Maps, Discrete/Continuous Kantorovich formulations, Wasserstein distances, and Entropic regularization.

Domain Adaptation Basics

Models trained on one dataset may not generalize well to unseen/unlabeled test sets. Consider this toy example. Our training set might consist of photos of North American cities during the summer, but there is poor generalization if we want to analyze photos of cities in the winter—the model has never seen snow before!

Abstractly, we want to know the joint $P(X, Y)$, but we only have empirical $\widehat{P}_{train}(X, Y)$. The issue at hand is if we learn predictor $f : X \to Y$ from $\widehat{P}_{train}(X, Y)$, this predictor could generalize poorly to empirical $\widehat{P}_{test}(X, Y)$ since our predictor only works on $\widehat{P}_{train}(X, Y)$. The train and test datasets might look very different (even though us humans might still be able to identify the same cities in the summer and winter).

Unsupervised Domain Adaptation

This is a common variant of DA:

- Labels are **only available in the source domain**.
- The classifier is tested on the **target domain**, where we have no labels.
- A classifier trained only on the source domain performs badly on the target domain.

Why is DA a Real Problem?

- People cannot afford to label all data from all sources/problems.
- **Synthetic data** constitutes a new domain. We would love to train a model on synthetic data (easy to label) and apply it to real data.
- **Remote Sensing Example:**
  - It is very time-consuming to label objects (e.g., cars, buildings) in large satellite images.
  - Domain shifts are created by: different geographic zones, atmospheric conditions, acquisition times, different sensor resolutions, or different data collection modes (e.g., LiDAR, RADAR).

Variants of Domain Adaptation

- **Unsupervised vs. Semi-Supervised DA:** Depends on whether a few labels are available in the target domain.
- **Heterogeneous DA:** Data from different domains do not lie in the same space (e.g., text and images).
- **Multi- vs. Single-Source DA:** Whether one or multiple source domains are available.
- **Covariate vs. Target Shift:**
  - **Covariate Shift:** The class-conditional distributions $\mathcal{P}(X|\text{label})$ are different.
  - **Target Shift:** The class proportions $\mathcal{P}(\text{label})$ are different in the target domain.
- **Domain Generalization:** Several source domains are available, but the target domain is not available during training. The goal is to generalize to an unseen target domain.
- **Source-Free DA:** Only a classifier trained on the source domain is available, not the source samples themselves.
- **Change in the Class Space:** The label space itself is changing.
  - **Closed Set DA:** Source and target classes are the same.
  - **Partial DA:** Target classes are a subset of source classes.
  - **Open Set DA:** Target domain contains classes not seen in the source.

State-of-the-Art (SOTA) Solutions for DA

The problem is how to learn a classifier that performs well on several domains when we only have labels in one domain.

- **Theory:** Measures the difficulty of the task as a function of the **discrepancy** of the representations of the data.
- **Solutions:**
  - Find a **domain-invariant representation** of the data (e.g., subspace projection, feature learning).
  - **Transform data** from one domain to another (e.g., adversarial methods).
  - **Divergence Measures:** Most methods involve a notion of divergence between the distributions:
    - Aligning 2nd order statistical moments (means and covariances).
    - Maximum Mean Discrepancy (MMD).
    - **Optimal Transport (OT)**.

2 Intro to Optimal Transport (OT)

The Origins of OT (Monge)

The original problem posed by Monge was how to move one mass distribution μ_s to another mass distribution μ_t while minimizing the total effort or cost.

- Find a mapping T between the two mass distributions.
- Minimize with respect to a displacement cost $c(x, y)$ that is the cost of moving mass from x to y.

The Kantorovich Formulation (1942)

Monge's problem is hard to solve. Kantorovich relaxed it by looking for a probabilistic **coupling** $\pi \in \mathcal{P}(\Omega_s \times \Omega_t)$, which is a joint probability distribution whose marginals are μ_s and μ_t.

The problem becomes a Linear Program:

$$\pi_0 = \arg\min_{\pi \in \Pi} \int_{\Omega_s \times \Omega_t} c(x, y)\pi(x, y)dxdy$$

subject to the marginal constraints:

$$\Pi = \{\pi \geq 0, \int_{\Omega_t} \pi(x, y)dy = \mu_s(x), \int_{\Omega_s} \pi(x, y)dx = \mu_t(y)\}$$

This Linear Program always has a solution.

Wasserstein Distance

The minimal cost of this transport is the Wasserstein distance. For a cost $c(x, y) = ||x - y||^p$, the p-Wasserstein distance is:

$$W_p^p(\mu_s, \mu_t) = \min_{\pi \in \Pi} \int_{\Omega_s \times \Omega_t} ||x - y||^p \pi(x, y)dxdy = \mathbb{E}_{(x,y)\sim\pi_0}[||x - y||^p]$$

Key properties:

- It does **not require the distributions to have overlapping support** (unlike KL divergence).
- It works for both continuous and discrete distributions (e.g., empirical histograms).
- Solved using Linear Program duals

Discrete OT

When dealing with discrete empirical distributions (e.g., n_s source samples and n_t target samples), the cost matrix is $\mathbf{C} \in \mathbb{R}^{n_s \times n_t}$ where $c_{ij} = c(x_i^s, x_j^t)$. The coupling π becomes a matrix π. The problem is:

$$\pi_0 = \arg\min_{\pi \in \Pi} \{\langle \pi, \mathbf{C} \rangle_F = \sum_{i,j} \pi_{ij} c_{ij}\}$$

where the marginal constraints are $\pi \mathbf{1}_{n_t} = \mu_s$ and $\pi^T \mathbf{1}_{n_s} = \mu_t$. This can be solved with a Network Flow solver in $O(n^3 \log n)$ time, which is computationally expensive for large n.

Regularized Optimal Transport

To make OT practical, we add a regularization term $\Omega(\pi)$. This b̈lurs the coupling.̈

$$\pi_0^\lambda = \arg\min_{\pi \in \Pi} \langle \pi, \mathbf{C} \rangle_F + \lambda \Omega(\pi)$$

The regularization term $\Omega(\pi)$ can be:

- **Entropic regularization:** $\Omega(\pi) = \sum_{i,j} \pi_{ij} \log \pi_{ij}$.
- **Group Lasso:** $\Omega_c(\pi) = \sum_j \sum_c ||\pi(\mathcal{I}_c, j)||_q^p$.
- Other divergences (KL, B-divergence, etc.).

Why regularize?

- **Faster algorithms:** Entropic regularization allows the use of the very fast **Sinkhorn-Knopp algorithm**. The solution has the form $\pi_0^\lambda = \text{diag}(u)\exp(-C/\lambda)\text{diag}(v)$.
- **Better problem properties:** The problem becomes strongly convex and the solution is more stable.
- **Smoother estimation:** It provides a smooth estimation of the (regularized) distance $W_\lambda(\mu_s, \mu_t)$.
- **Prior knowledge:** Regularization (like Group Lasso) can encode prior knowledge into the transport plan.

3 Optimal Transport for Domain Adaptation via Barycentric Mapping

This was an early approach to using OT for DA.

Assumptions

1. There exists a transport mapping T in the feature space between the two domains.
2. The transport **preserves the conditional distributions** (i.e., the labels):

$$P_s(y|X_s) = P_t(y|T(X_s))$$

The 3-Step Strategy [Courty et al., 2016b, PAMI]

1. **Estimate OT:** Compute the optimal transport coupling π_0 between the source μ_s and target μ_t distributions.
2. **Transport Samples:** Transport the source training samples X_s to the target domain using **barycentric mapping**.
3. **Learn Classifier:** Learn a standard classifier f on the *transported* source samples and their original labels $\{(T(x_i^s), y_i^s)\}_{i=1}^{n_s}$.

Barycentric Mapping [Ferradans et al., 2014]

The barycentric mapping transports a single source sample x_i^s by spreading its mass onto all target samples according to the i-th row of the coupling matrix π_0.

The transported sample $T(x_i^s)$ is the barycenter of the target samples, weighted by the coupling:

$$T(x_i^s) = \arg\min_x \sum_{j=1}^{n_t} \pi_0(i,j)c(x, x_j^t)$$

There is a closed-form quadratic cost! But we are limited to in-distribution samples.

Class-Based Regularization

A problem with the simple 3-step strategy is that the transport plan π_0 might map samples from one class to another. We can penalize cross-class mappings.

We want the transport coupling to **only link samples that are in the same class**. We can enforce this using a **Group Lasso** regularization term:

$$\Omega_c(\pi) = \sum_{j=1}^{n_t}\sum_{c=1}^{K} ||\pi(\mathcal{I}_c, j)||_q^p$$

- $\mathcal{I}_c$ contains the indices (rows) of the source samples X_s that belong to class c.
- When $p \leq 1$, this regularization encourages the columns of π (i.e., each target sample j) to be sparse in terms of the *groups* of classes it's connected to.
- This encourages each target sample to receive mass from **only one source class**.

Limitations of this Approach

- It models the transformation $T : \mathbb{R}^d \to \mathbb{R}^d$ but our end goal is just a classifier $f : \mathbb{R}^d \to \mathbb{R}$.
- It **requires the same class proportions** between domains.

4 OT on Joint Distributions (JDOT)

This approach is more direct. Instead of transporting features and then training a classifier, JDOT learns the classifier *as part* of the transport problem. It models the transformation of labels and allows for changes in class proportions. It learns the optimal target predictor f directly, even with no target labels.

The JDOT Formulation [Courty et al., 2017, NIPS]

We work with the joint feature/label distributions $\mathcal{P}_s(X, Y)$ and $\mathcal{P}_t(X, Y)$.

- We have the empirical source distribution: $\hat{\mathcal{P}}_s = \frac{1}{N_s} \sum_{i=1}^{N_s} \delta_{(x_i^s, y_i^s)}$.
- We only have features for the target: $X_t = \{x_j^t\}_{j=1}^{N_t}$.

We define a proxy joint distribution for the target domain using a classifier f:

$$\hat{\mathcal{P}}_t^f = \frac{1}{N_t} \sum_{j=1}^{N_t} \delta_{(x_j^t, f(x_j^t))}$$

The **JDOT problem** is to find the predictor f that best aligns these two joint distributions:

$$\min_{f \in \mathcal{H}} \{\mathcal{W}_1(\hat{\mathcal{P}}_s, \hat{\mathcal{P}}_t^f)\} = \min_{f \in \mathcal{H}} \left\{ \inf_{\pi \in \Delta} \sum_{i,j} \mathcal{D}((x_i^s, y_i^s); (x_j^t, f(x_j^t)))\pi_{ij} \right\}$$

where Δ is the transport polytope. And $\mathcal{D}(\cdot, \cdot)$ defines a special distance in the joint space:

$$\mathcal{D}((x_i^s, y_i^s); (x_j^t, f(x_j^t))) = \underbrace{\alpha ||x_i^s - x_j^t||^2}_{\text{Feature distance}} + \underbrace{\mathcal{L}(y_i^s, f(x_j^t))}_{\text{Label/Loss distance}}$$

Optimization

The full optimization problem adds a regularization $\Omega(f)$ for the predictor f:

$$\min_{f \in \mathcal{H}, \pi \in \Delta} \sum_{i,j} \pi_{ij} \left(\alpha d(x_i^s, x_j^t) + \mathcal{L}(y_i^s, f(x_j^t)) \right) + \lambda \Omega(f)$$

This is solved using **Block Coordinate Descent (BCD)**, alternating between updating π and f.

1. **Update π (for fixed f):** The cost matrix $\mathbf{C}_{ij} = \alpha d(x_i^s, x_j^t) + \mathcal{L}(y_i^s, f(x_j^t))$ is now fixed. This is just a **classical OT problem**, which can be solved with a network simplex or Sinkhorn (if entropically regularized).
2. **Update f (for fixed π):**

   $$\min_{f \in \mathcal{H}} \sum_{i,j} \pi_{ij} \mathcal{L}(y_i^s, f(x_j^t)) + \lambda \Omega(f)$$

   This is a standard **weighted loss** supervised learning problem. The coupling matrix π **performs label propagation**.

Label Propagation

We can consider the classes in our source domain as belonging to one-hot-encoded classes, and then our transport plan gives us a way of translating these mappings to the target domain. Specifically, we can consider $\hat{y}_j = N_t \sum_i \pi_{ij} y_i^s$ as the weighted average of source labels propagated to target sample j.

With this remapping, the f-update step can be interpreted as training f on the target samples x_j^t with soft propagated labels $\hat{y}_j$.

For least-squares regression, the f-update becomes:

$$\min_{f \in \mathcal{H}} \sum_{j=1}^{N_t} \frac{1}{N_t} ||\hat{y}_j - f(x_j^t)||^2 + \lambda ||f||^2$$

For multiclass classification with Hinge loss, the f-update becomes a weighted SVM problem where the weights are derived from the propagated class proportion matrix $\hat{P} = \frac{1}{N_t} \pi^T P^s$.

Impact of Class Proportion (Target Shift)

A major limitation of early methods was the assumption that $\mathcal{P}_s(\text{label}) = \mathcal{P}_t(\text{label})$.

- If target proportions are **known**, we can re-weight the source samples to match.
- If target proportions are **unknown**, standard OT (even JDOT) can fail, as it will mix instances from different classes.

JCPOT [Redko et al., 2019]

JCPOT **simultaneously estimates the target class proportions** Δ by casting the problem as finding the Wasserstein barycenter of the source domains:

$$\min_{\Delta} \sum_{k=1}^{K} \mathcal{W}_\rho^\rho(\mu_s^{(k)}, \mu_t^\Delta)$$

By estimating the target proportions, JCPOT results in less mixing of classes in the transport plan.

Deep JDOT

Modern DA aims to align source and target domains within a **deep representation space**.

How to scale JDOT to deep learning and large datasets?

1. **Learn the embedding [Genevay et al., 2017]:** Simultaneously learn the feature extractor g_θ (the backbone network) that makes the alignment easiest. The distance $d(x_i^s, x_j^t)$ is replaced with $d(g_\theta(x_i^s), g_\theta(x_j^t))$.
2. **Minibatching:** Compute the transport plan not on the full dataset, but between minibatches from the source and target domains.

The **Deep JDOT Algorithm** follows the same BCD structure:

- **for** BCD Iterations **do**
  - *# π-update step (on minibatches)*
  - **for** each Source/Target minibatch **do**
    - Solve the OT problem on the minibatch (using the current g_θ and f) to get a local π.
    - Perform label propagation on the minibatch using π.
  - **end**
  - *# f and g_θ update step*
  - Update the model (both g_θ and f) on one epoch using the propagated labels from all minibatches.
- **end**

Minibatch OT (MBOT)

The idea of computing OT between minibatches (b_s, b_t) is simple:

$$MBOT_p^p(\mu_s, \mu_t) := \mathbb{E}_{(X,Y)\sim\mu_s^{\otimes m}\otimes\mu_t^{\otimes m}}[\mathcal{W}_p^p(b_s, b_t)]$$

where $b_s = \frac{1}{m}\sum_i \delta_{x_i}$ and $\underline{t} = \frac{1}{m}\sum_i \delta_{y_i}$.

This formulation preserves marginals while providing some level of regularization. Fatras et al., 2020 and Fatras et al., 2021b show interesting asymptotic properties and interchange between sums and gradients respectively.

However, this formulation also introduces **sampling effects**. The small batch size can create bad matchings and the estimation is noisy. Mitigate this with unbalanced OT.

JUMBOT: Unbalanced Minibatch OT

The solution is to **relax the marginal constraints** $\pi\mathbf{1} = \mu_s$ and $\pi^T\mathbf{1} = \mu_t$. This is called **Unbalanced Optimal Transport (UOT)**.

UOT allows the transport plan to *ignore* some mass (e.g., outliers created by sampling) by paying a penalty τ.

$$UOT^{\tau,\epsilon}(\alpha, \beta) = \min_{\pi\in M_+(\mathcal{X}\times\mathcal{Y})} \int cd\pi + \epsilon KL(\pi|\alpha\otimes\beta) + \tau(kL(\pi_1||\alpha) + kL(\pi_2||\beta))$$

JUMBOT (Joint Unbalanced Minibatch Optimal Transport) simply replaces the standard OT solver in Deep JDOT with an Unbalanced OT solver.

Advantages:

- Mitigates the sampling effects of small batches.
- The target shift impact is reduced.
- Naturally handles **Partial Domain Adaptation** (where target classes are a subset of source classes).

References

1. Courty, N., Flamary, R., Tuia, D., and Rakotomamonjy, A. (2016b). Optimal transport for domain adaptation. In *Pattern Analysis and Machine Intelligence, IEEE Transactions on*.
2. Courty, N., Flamary, R., Habrard, A., and Rakotomamonjy, A. (2017). Joint distribution optimal transportation for domain adaptation. In *Neural Information Processing Systems (NIPS)*.
3. Fatras, K., Zine, Y., Flamary, R., Gribonval, R., and Courty, N. (2020). Learning with minibatch wasserstein: asymptotic and gradient properties. In *AISTATS*
4. Fatras, K., Zine, Y., Majewski, S., Flamary, R., Gribonval, R., and Courty, N. (2021b). Minibatch optimal transport distances; analysis and applications.
5. Ferradans, S., Papadakis, N., Peyré, G., and Aujol, J.-F. (2014). Regularized discrete optimal transport. In *SIAM Journal on Imaging Sciences, 7(3)*.
6. Genevay, A., Peyré, G., and Cuturi, M. (2017). Sinkhorn-autodiff: Tractable wasserstein learning of generative models. *arXiv preprint arXiv:1706.00292*
7. Redko, I., Courty, N., Flamary, R., and Tuia, D. (2019). Optimal transport for multi-source domain adaptation under target shift. In Chaudhuri, K. and Sugiyama, M., editors, *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of Proceedings of Machine Learning Research, pages 849–858. PMLR.